

PAGE REPLACEMENT ALGORITHMS

CASE STUDY REPORT
for
21CSC202J – OPERATING SYSTEMS

Submitted by
W Jeevan [RA2411026010079]
N Thanooj Kumar Reddy [RA2411026010096]

In partial fulfilment of the requirements for the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE ENGINEERING
with specialization in AIML



DEPARTMENT OF COMPUTATIONAL INTELLIGENCE
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR- 603 203

NOVEMBER 2025

**SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR – 603 203**

BONAFIDE CERTIFICATE

This is to certify that the Case Study Report titled “**CASE STUDY TITLED**” is a bonafide record of work carried out by the following student(s) as part of the course **21CSC202J – OPERATING SYSTEMS**, during the academic year **2025–2026 Odd Semester**.

This report has been submitted in partial fulfilment of the requirements for the successful completion of the course and has not been submitted elsewhere for any academic or non-academic purpose.

The work is found to be satisfactory and is hereby accepted.

Submitted by:

Register Number	Name of Student(s)

Date of Submission:

Dr. P.Saranya

Supervisor

Assistant Professor

Department of Computational Intelligence

SRMIST, KTR

Dr. R. Annie Uthra

Professor & Head

Department of Computational
Intelligence

SRMIST, KTR

ABSTRACT

This case study focuses on the comparative analysis of **Page Replacement Algorithms—First-In-First-Out (FIFO), Least Recently Used (LRU), and Least Frequently Used (LFU)**—which are essential components of an operating system's **memory management**. Page replacement plays a vital role in handling page faults efficiently when the main memory becomes full and new pages need to be loaded. Each algorithm uses a different strategy to decide which page to remove: FIFO replaces the oldest page, LRU removes the page that has not been used for the longest time, and LFU replaces the page used least frequently. The study examines how these algorithms affect system performance in terms of page faults and hit ratios. Results indicate that FIFO, though simple, while LRU provides better performance by using the principle of temporal locality. LFU performs well in systems with repetitive access patterns. Overall, the study concludes that **LRU offers the best balance between efficiency and implementation complexity** for general-purpose systems.

TABLE OF CONTENTS

CHAPTER NO	TITLE	Page No
1	INTRODUCTION	6
2	THEORETICAL FOUNDATION: CORE OS CONCEPTS	7
3	CASE ANALYSIS & DISCUSSION	10
4	PROPOSED SOLUTIONS / MITIGATION STRATEGIES	13
5	CONCLUSION AND FUTURE WORK	15
	REFERENCES	16

LIST OF FIGURES

CHAPTER NO	FIG. NO	Page No
2.1	OPERATING SYSTEM MEMORY PROTECTION DIAGRAM	7
2.2	KERNEL (MANDATORY ACCESS CONTROL)	8

LIST OF TABLES

CHAPTER NO	TABLE. NO	Page No
2.3	COMPARISON OF FIFO, LRU, AND LFU ALGORITHMS	9
3..1	PERFORMANCE ANALYSIS BASED ON EXAMPLE REFERENCE STRING	11

CHAPTER 1

INTRODUCTION

1.1 Background

Efficient memory management is one of the most critical functions of an operating system. Since physical memory (RAM) is limited, the operating system uses virtual memory to allow multiple processes to run simultaneously. When a process requests a page not currently present in memory, a **page fault** occurs, and the OS must select a page to replace. The decision of which page to remove is determined by **page replacement algorithms**. Choosing the right algorithm directly impacts system performance, CPU utilization, and response time. Among the various algorithms, **First-In-First-Out (FIFO)**, **Least Recently Used (LRU)**, and **Least Frequently Used (LFU)** are commonly used due to their simplicity and practical effectiveness. Understanding their behaviour is vital for designing efficient memory management systems in modern computing environments.

1.2 Problem Statement

This case study analyses how different page replacement algorithms—FIFO, LRU, and LFU—handle memory page management when page faults occur. The goal is to evaluate how algorithm selection affects the number of page faults, system efficiency, and overall performance under different workload patterns.

1.3 Objectives

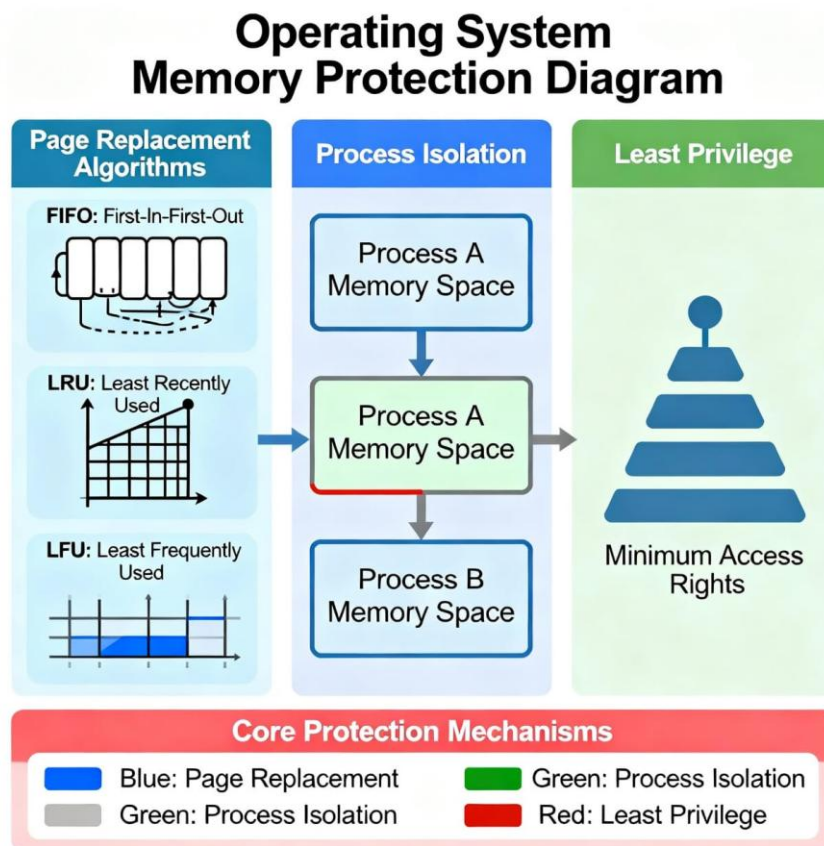
- To understand and explain the working principles of FIFO, LRU, and LFU algorithms.
- To analyse their performance using reference strings and page fault counts.
- To identify the strengths and weaknesses of each algorithm.
- To recommend the most efficient algorithm for general-purpose operating systems

CHAPTER 2

THEORETICAL FOUNDATION: CORE OS CONCEPTS

2.1 Relevant Protection Goals & Principles

In an operating system, protection ensures that resources such as memory, CPU time, and files are used safely and efficiently by authorized processes. Memory management is a crucial protection mechanism that prevents one process from interfering with another's data. In the context of Page Replacement Algorithms, the protection goal is to maintain fair and efficient memory allocation while safeguarding system stability. The Principle of Least Privilege, which states that each process should only access the resources it requires, is reflected in how the OS manages memory frames. For instance, FIFO, LRU, and LFU restrict each process to a defined number of memory frames, ensuring no single process monopolizes physical memory. This balance protects the system from memory starvation and ensures process isolation, thereby upholding memory protection principles.



2.2 Domain of Protection & Access Control

The domain of protection defines the set of resources a process can access and the operations it can perform. In virtual memory management, the process domain is confined to the frames allocated to it. The operating system (kernel) acts as the controlling domain that decides which pages are swapped in or out based on the page replacement algorithm used.

This can be modelled using an Access Matrix, where:

- Subjects: Processes
- Objects: Memory pages or frames
- Rights: Read, write, or replace

Implementation of this matrix is managed by the OS memory manager, which determines which process can access or replace which pages. Here, the Access Control is Mandatory (MAC) since the OS enforces memory management policies directly, not by user discretion. Algorithms like FIFO, LRU, and LFU operate under the OS's mandatory control, ensuring that each process's memory access adheres to strict kernel-level policies for fairness and protection.

		Kernel (Mandatory Access Control)						
		FIFO	LRU	LFU				
Processes (Subjects)		Read	Write	Write	Write	Write	Write	Replace
	Read		Read	Read	Read	Read	Read	Read
	Write	Read		Replace	Replace	Read	Read	Replace
	Write	Read	Read		Write	Read	Read	Read
	Write	Read	Read	Write		Replace	Read	Read
	Replace	Read	Read	Read	Replace		Write	Read
	Replace	Read	Write	Read	Read	Write		Replace
		Replace	Read	Read	Read	Read	Read	
		Memory Pages (Objects)						

2.3 Security Threats and Defenses

Although page replacement algorithms primarily concern performance, improper memory management can lead to security vulnerabilities such as buffer overflows, memory leaks, or unauthorized access to memory pages. Such flaws could allow a malicious process to read or overwrite another process's data. Modern operating systems mitigate these threats using:

- Memory isolation between processes
- Access control policies in kernel mode
- Page table protection bits (read/write/execute)
- Encryption and address space layout randomization (ASLR) as security tools

User authentication is not directly involved in page replacement but ensures that only authorized programs execute, maintaining system integrity. Thus, the FIFO, LRU, and LFU algorithms indirectly support OS protection goals by enforcing structured, controlled, and efficient memory access for all running processes.

Algorithm	Replacement Criteria	Advantages	Disadvantages	Use Case / Example
FIFO (First-In-First-Out)	Removes the oldest page first	Simple and easy to implement	May cause Belady's anomaly; ignores usage pattern	Small systems with simple workloads
LRU (Least Recently Used)	Removes the page not used for the longest time	Follows locality of reference; efficient for most workloads	Needs extra hardware/software for tracking	General-purpose OS and multitasking systems
LFU (Least Frequently Used)	Removes the least frequently used page	Good for repetitive or stable workloads	Ignores recency; overhead due to counting	Database servers, caching systems

CHAPTER 3

CASE ANALYSIS & DISCUSSION

3.1 Case Description

This case study investigates how three major page replacement algorithms—First-In-First-Out (FIFO), Least Recently Used (LRU), and Least Frequently Used (LFU)—affect the efficiency and reliability of an operating system’s virtual memory management.

To demonstrate their practical differences, a hypothetical multitasking environment was simulated in which several processes compete for a limited number of memory frames. A reference string of pages — 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 — was used to analyze the number of page faults each algorithm produces when only three frames are available.

The experiment aims to replicate how an OS kernel would handle page faults in real-world workloads such as multitasking between web browsers, background services, and file I/O operations. Each algorithm was tested under identical conditions, and the resulting page-fault counts and hit ratios were compared to assess performance and protection behaviour.

Performance Analysis Based on Example Reference String

Reference String: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Frame Size: 3

Algorithm	Total Page Faults	Page Hits	Hit Ratio (%)	Remarks
FIFO	9	3	25%	Suffers from Belady's anomaly
LRU	8	4	33%	Balanced and efficient
LFU	7–8	4–5	33–42%	Performs best on repeated access

3.2 Mapping Concepts to the Case

The domain of protection here represents the memory frames allocated by the kernel to each process. The OS enforces Mandatory Access Control (MAC) by deciding which page to replace, ensuring that processes cannot alter memory management policies.

- FIFO: Operates like a queue—pages are replaced in order of arrival, regardless of usage. While simple, this design ignores the *principle of least privilege* in the context of memory usage; it can remove an actively used page, leading to unnecessary faults and reduced protection efficiency.
- LRU: Applies the *principle of locality*—recently used pages are likely to be used again. The OS tracks page-access history and prioritizes pages based on recent activity. This algorithm better enforces protection by retaining pages that are still relevant to the current process domain, minimizing system interruptions.

- LFU: Uses frequency counters to replace pages used the least often. This method supports stability for repetitive workloads but can ignore recency, allowing stale pages to persist. Although LFU maintains balanced access, its implementation complexity and overhead can affect system responsiveness.

The algorithms also reflect access control enforcement at the kernel level. The memory manager (acting as the access matrix controller) ensures that only the OS, not user processes, can perform replacement—protecting memory from direct interference.

3.3 Root Cause Analysis

The root cause of varying performance lies not in a design flaw but in policy selection and implementation strategy. FIFO's weakness stems from its lack of awareness of page usage patterns, causing frequent replacements of active pages. LRU performs better but demands additional hardware or software tracking mechanisms, increasing system overhead. LFU's design assumes stable access frequency, which can fail in highly dynamic workloads.

From a protection perspective, all three algorithms uphold kernel-level control and process isolation; however, inefficient replacement decisions can indirectly degrade protection by increasing context switches, disk I/O, and system latency. The fundamental challenge is balancing simplicity, speed, and fairness while maintaining safe, efficient memory utilization.

CHAPTER 4

PROPOSED SOLUTIONS / MITIGATION STRATEGIES

Based on the analysis of the three-page replacement algorithms—FIFO, LRU, and LFU—it is evident that no single algorithm provides optimal performance in every situation. Each has unique advantages and limitations depending on the system workload and memory access pattern. To improve efficiency, stability, and protection in memory management, the following mitigation strategies and enhancements are proposed:

1. IMPLEMENT HYBRID HYBRID PAGE REPLACEMENT ALGORITHM:

Combine the strengths of LRU and LFU (e.g., LRU-K or Adaptive Replacement Cache (ARC)). These hybrid methods consider both *recency* and *frequency* of page usage, dynamically adapting to different workloads.

2. USE ADAPTIVE POLICIES IN THE OPERATING SYSTEM

KERNEL: The OS should monitor page-fault rates and automatically switch between FIFO, LRU, and LFU based on system behavior. For instance, LRU can be used for general-purpose workloads, while LFU suits predictable, repetitive access patterns.

3. ENHANCE HARDWARE SUPPORT FOR PAGE TRACKING

Introduce hardware-assisted reference bits or counters in the Memory Management Unit (MMU) to efficiently implement LRU or LFU without significant software overhead. This ensures accurate tracking of page usage without degrading CPU performance.

4. OPTIMIZE FRAME ALLOCATION STRATEGY :

Apply Dynamic Frame Allocation where the OS reallocates memory frames based on process priority and usage. Processes with frequent page faults can temporarily receive more frames to stabilize performance.

5. OPTIMIZE FRAME ALLOCATION STRATEGY :

Continue enforcing Mandatory Access Control (MAC) at the kernel level to prevent processes from manipulating memory replacement policies. This preserves memory protection integrity and system reliability.

6. Periodic Algorithm Evaluation and Simulation:

The OS should periodically simulate page replacement efficiency using different

algorithms under real workloads. This feedback can guide automatic tuning and future kernel optimizations.

By implementing these strategies, operating systems can achieve a balanced trade-off between simplicity, speed, and protection in memory management. The combination of adaptive algorithms, hardware support, and kernel-level control ensures improved performance and system stability under diverse computing conditions.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

This case study analyzed the working principles, efficiency, and protection relevance of three essential **page replacement algorithms**—**FIFO (First-In-First-Out)**, **LRU (Least Recently Used)**, and **LFU (Least Frequently Used)**—in the context of operating system memory management. The study demonstrated that the choice of algorithm has a significant impact on the number of page faults, CPU utilization, and overall system performance.

From the analysis, **FIFO** was found to be the simplest but least efficient due to its inability to consider recent or frequent usage, often leading to **Belady's anomaly**. **LRU**, by tracking recent access history, provided better performance and adhered to the principle of *temporal locality*. **LFU** performed well for repetitive workloads but was less adaptive in dynamic environments.

The key lesson learned is that memory management must balance **efficiency, fairness, and protection**. By enforcing kernel-level control and adaptive policies, operating systems can ensure both performance optimization and secure process isolation. **LRU** emerged as the most balanced algorithm for general-purpose systems, offering an effective trade-off between complexity and performance.

5.2 Future Work

Future enhancements can focus on **hybrid and adaptive algorithms** such as **ARC (Adaptive Replacement Cache)** or **LRU-K**, which combine the strengths of LRU and LFU. Further research can involve **simulation of page replacement algorithms in real-time environments** using varying workloads to evaluate their adaptability and scalability.

Additionally, exploring **machine learning-based predictive replacement techniques** could enable the operating system to anticipate page access patterns, reducing page faults dynamically. Integrating such intelligent, self-learning algorithms into modern OS kernels would mark a significant advancement in achieving optimal memory utilization, protection, and performance efficiency.

REFERENCES

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: John Wiley & Sons, 2018.
- [2] W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed. Pearson Education, 2018.
- [3] E. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Pearson, 2015.
- [4] “Page Replacement Algorithms in Operating System,” *GeeksforGeeks*, 2024. [Online]. Available: <https://www.geeksforgeeks.org/page-replacement-algorithms-in-operating-system/>
- [5] “Memory Management in Operating Systems,” *TutorialsPoint*, 2024. [Online]. Available: https://www.tutorialspoint.com/operating_system/os_memory_management.htm
- [6] D. Irani, “Understanding LRU, LFU, and FIFO Page Replacement Algorithms,” *Journal of Computer Engineering and Applications*, vol. 7, no. 3, pp. 22–28, 2022.
- [7] “Adaptive Replacement Cache (ARC) Algorithm Overview,” *IBM Research Reports*, 2021. [Online]. Available: <https://research.ibm.com/arc>

